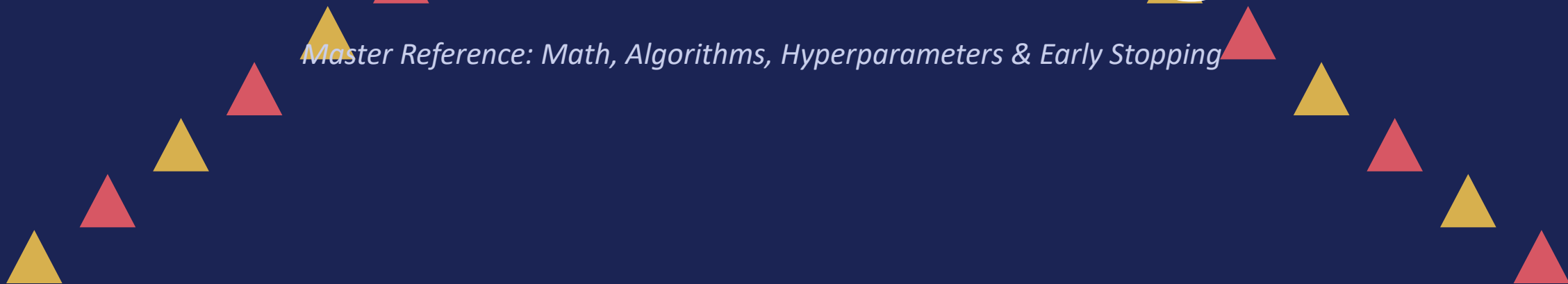


ENSEMBLE LEARNING · SEQUENTIAL TREE BOOSTING

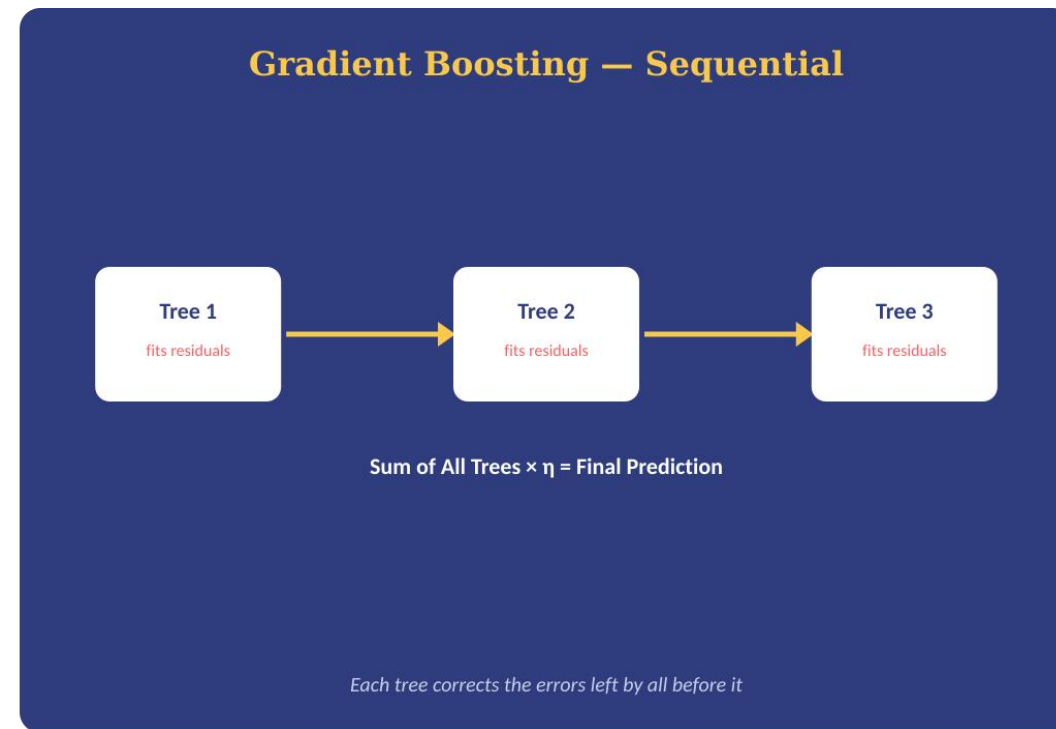
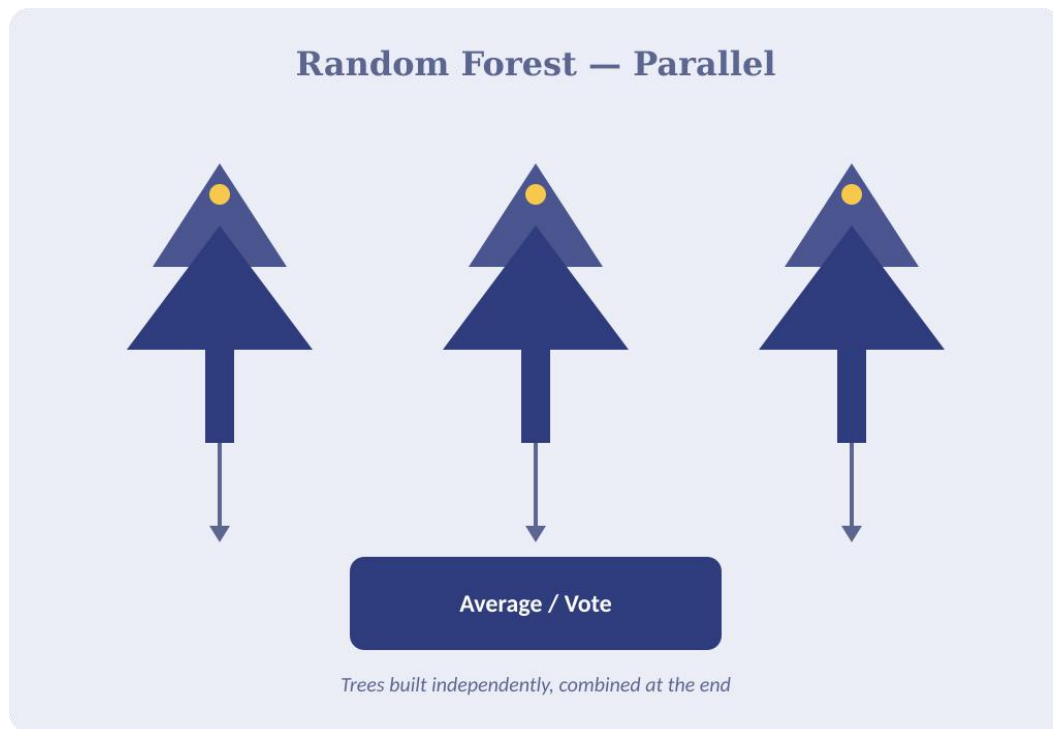
Gradient Boosting

Master Reference: Math, Algorithms, Hyperparameters & Early Stopping



Core Concept: Sequential Tree Boosting

An **ensemble** of weak, shallow decision trees (depth 2–6), trained one after another. Unlike Random Forests, which build trees **independently in parallel**, each new Gradient Boosting tree is trained explicitly to correct the **residual errors** left by every tree before it.



Regression Algorithm — Step by Step

Predicting continuous values, e.g. patient recovery days or price



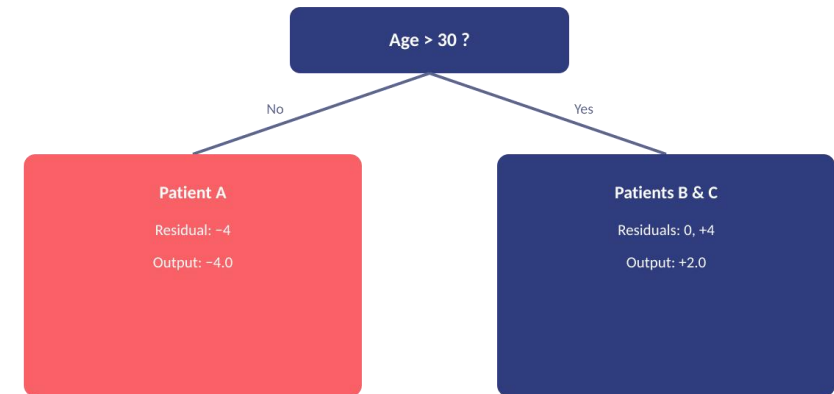
$$F0 = (1/N) \sum y_i \quad r = y - \hat{y} \quad \text{Leaf Output} = \sum \text{residuals} / N_{\text{leaf}} \quad \hat{y}_{\text{new}} = \hat{y}_{\text{old}} + \eta \times \text{Leaf Output}$$

Node split metric: minimize Mean Squared Error (MSE) across left/right child nodes when choosing the best feature split.

Solved Example: Patient Recovery Days

Learning rate $\eta = 0.1$

Patient	Age	Actual	$r_1 = y - F_0$	Pred after Tree 1	r_2 (new)
A	20	6	-4	9.6	-3.6
B	40	10	0	10.2	-0.2
C	60	14	+4	10.2	+3.8



$$F_0 = (6 + 10 + 14) / 3 = 10 \text{ days}$$

Reading the Result

- Prediction 1 = $10 + (0.1 \times \text{Leaf Output})$
- Patient A (Leaf 1, output -4): $10 + (0.1 \times -4) = 9.6$ days
- Patients B & C (Leaf 2, output +2): $10 + (0.1 \times 2) = 10.2$ days

The tiny η step nudges predictions toward the truth without overreacting — residuals shrink and feed straight into Tree #2.

Classification Algorithm — Step by Step

Predicting binary labels, e.g. sick (1) vs healthy (0)



Node split metric: minimize Log-Loss (Binary Cross-Entropy) · Leaf output uses a Newton-Raphson step since probabilities are bounded 0–1

Solved Example: Patient Status

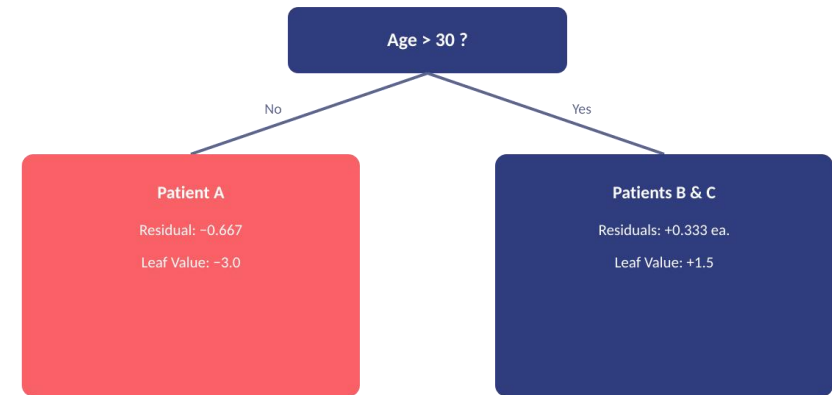
Learning rate $\eta = 0.1$

Patient	Age	Target	$r_1 = y - p_0$	New p_1	r_2 (new)
A	20	0	-0.667	0.597	-0.597
B	45	1	+0.333	0.699	+0.301
C	60	1	+0.333	0.699	+0.301

Reading the Result

- Leaf 1 (A): $-0.667 / [0.667 \times 0.333] = -3.0$
- Leaf 2 (B, C): $0.666 / [0.222 + 0.222] = +1.5$
- New Log-Odds = $0.693 + (0.1 \times \text{leaf value})$

Predictions update in log-odds space, then convert back to probability via sigmoid before the next tree begins.



$$F_0 = \ln(2/1) \approx +0.693 \rightarrow p_0 = \text{sigmoid}(0.693) \approx 0.667$$

Regression vs. Classification

Same sequential-correction philosophy, different math underneath

Component	Regression GBDT	Classification GBDT
Initial Guess (F0)	Arithmetic mean ($\bar{y}$)	Log-odds: $\ln(\text{Count1} / \text{Count0})$
Prediction Scale	Real numerical target units	Log-odds, converted via sigmoid
Residual Formula	$y_i - \hat{y}_i$	$y_i - p_i$
Node Split Metric	MSE minimization	Log-Loss (cross-entropy) minimization
Leaf Output Formula	$\sum r_i / N_{\text{leaf}}$ (simple average)	$\sum r_i / \sum [p_i(1-p_i)]$ (Newton-Raphson)

Where It Excels — And Where It Struggles



Suitable Data

- Tabular / structured data — industry gold standard
- Mixed numerical + categorical features
- No feature scaling required
- Complex, non-linear feature interactions



Data to Avoid

- Unstructured data — images, audio, video, raw text
- High-dim sparse text (bag-of-words) — use linear models
- Tiny datasets under ~100 rows — use Ridge / Logistic Regression



Limitations

- No extrapolation — can't predict beyond training range
- Sequential bottleneck — trees can't fully parallelize
- Outlier sensitivity — later trees chase noisy residuals

Key Hyperparameters



learning_rate (η)

Step size / contribution of each tree

Typically 0.01 – 0.1



n_estimators

Maximum number of sequential trees built

Set high, let early stopping cut it off



max_depth

Depth of each individual weak tree

Typically 3 – 6



subsample

Fraction of rows randomly sampled per tree

e.g. 0.8

Validation Architecture & Early Stopping

Data Split

Training Set — 80%

Validation
Set — 20%



early_stopping_rounds = N is a patience counter: after every tree, check the validation score. If it fails to beat its best record for N consecutive trees, halt training and roll back to the tree with the best validation score.

Learning Rate & Early Stopping: The Tradeoff



High LR (e.g. 0.2)

Big strides — learns fast but bounces past the valley floor. Fluctuations trigger early stopping before reaching true optimal accuracy.

Low LR (e.g. 0.01)

Small, precise steps that avoid overshooting and settle into the true minimum. Needs hundreds of trees to converge.

The Golden Model-Building Workflow

1

Set a low learning rate

e.g. $\eta = 0.01$ — for maximum precision and minimal overshoot

2

Set a large tree count ceiling

e.g. `n_estimators = 2000` — give it room to converge

3

Turn on early stopping

e.g. `early_stopping_rounds = 10–15` — auto-cuts training the moment overfitting begins

Result: the low learning rate guarantees high accuracy, while early stopping cuts training the exact moment overfitting begins — saving time without guessing the tree count.

Thank You